

ModelForge

Compiler Design Phase 2

Name: D.N.S.Vaishnavi

Reg no: 24BCB0093

Abstract

Phase 2 turns the ModelForge design into a working compiler pipeline. It reads a small model description, checks its tensors and operations, builds an intermediate representation, applies optimizations, and produces standalone C++ inference code. The compiler also prints its internal stages so the translation can be explained during a demonstration.

The default demonstration uses a dependency-free .mforge file for a small Iris-style network. This model has illustrative weights; it is not presented as a trained Iris classifier. ONNX loading and ONNX Runtime comparison are optional build paths, not requirements of the default demonstration.

1. Phase 2 Objective

The aim of this phase was to implement the main compiler modules and show that they work together. The current system handles static float32 feed-forward graphs with Gemm, MatMul, Add, Relu, Sigmoid and Softmax. It rejects unsupported models with a diagnostic instead of silently generating incorrect code.

2. Compiler Workflow

A model moves through five modules. The loader creates a ModelGraph; the validator builds a tensor symbol table; the IR builder makes the graph independent of its input format; the optimizer changes the IR; and the code generator writes model.cpp and model.h. The generated program is built separately and accepts either one input vector or a CSV file of input rows.

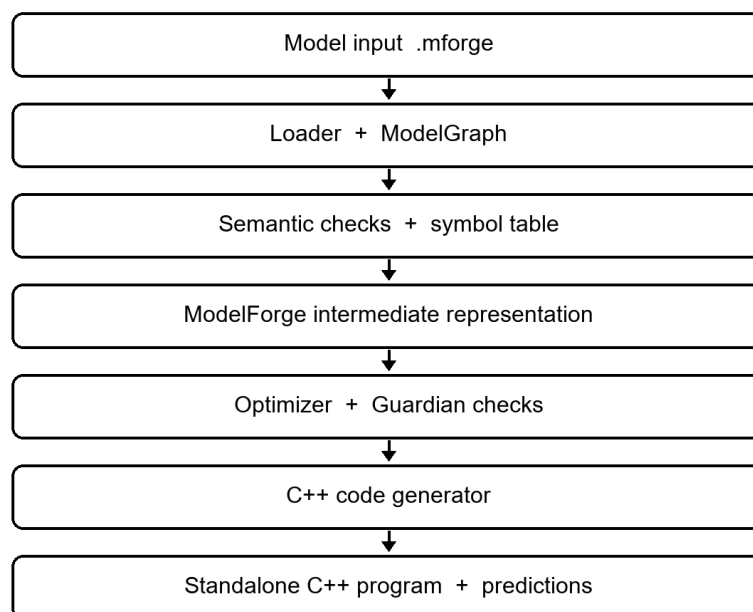


Figure 1. ModelForge Phase 2 compilation flow.

3. Core Implementation

3.1 Model loader

The front end reads each model, input, output, tensor and node declaration in a .mforge file. It recognizes operator names, tensor names, shapes, constant values and attributes such as the Softmax axis. These records become a ModelGraph. This is the parsing stage for the text manifest; ONNX itself is already a structured graph format.

3.2 Semantic checks and symbol table

The validator records each tensor's name, type, shape, source and constant status. Before later stages run, it checks that node inputs exist, output names are not duplicated, matrix dimensions agree, data types are supported and Softmax uses a valid axis. For example, a missing tensor is reported during validation rather than becoming a failure in generated C++.

3.3 Intermediate representation

The IR stores one instruction per operation with its input and output tensor names. It is easier to inspect and optimize than the original model description. For the demonstration network, the main path is:

```
INPUT x -> GEMM -> hidden1 -> RELU -> hidden2
GEMM hidden2 -> scores -> SOFTMAX -> y -> RETURN
```

The compiler prints this form before and after optimization, including an intentionally unused branch in the input graph.

3.4 Optimization

Three passes are implemented: constant folding for calculations with known inputs, fusion of a dense operation followed by ReLU, and removal of nodes that cannot affect the model output. On the sample graph, fusion and dead-node removal reduce seven IR instructions to five. Constant folding is implemented and tested separately, but this graph has no foldable operation.

3.5 C++ generation and execution

The back end emits readable C++17 with embedded weights, tensor operations and an infer function. The generated program can run a zero-input demo, accept four values on its command line, or process a numeric CSV row by row. It prints class scores and the predicted class. The IR interpreter is also run during compilation as a local execution check.

4. Checking Optimization Changes

A wrong optimization may change a prediction without causing a build error. ModelForge's Guardian tests a candidate graph against the preceding graph using fixed, random and graph-aware input probes. A pass is accepted only when the checked outputs agree within a numerical tolerance; a detected mismatch causes that pass to be rejected and saves an input that reproduces it.

For the review demonstration, an intentionally incorrect ReLU-to-Sigmoid rewrite is rejected. The normal generated model is built from the accepted graph, not the deliberately broken candidate. The optimization certificate records the pass decisions and comparison results. These finite probes are useful checks, but they are not a mathematical proof that every possible input is equivalent.

5. Functional Demonstration

The VS Code terminal can build the compiler and run the sample model with the following commands:

```
cmake -S . -B build
cmake --build build --config Release
ctest --test-dir build -C Release --output-on-failure
$model = '.\ModelForge\review_2_core_implementation\models\iris_demo.mforge'
.\build\Release\modelforge.exe $model --out .\build\generated --guardian-demo-bug
```

The program prints the parsed model tree, symbol table, IR, optimization report and final prediction, then writes model.cpp, model.h and optimization_certificate.json. After building the generated source, input 1 2 3 4 produces scores about 0.2404, 0.5568 and 0.2028, so the predicted class is 1. The zero-input demo predicts class 2. A supplied two-row CSV produces one result for each row.

6. Testing and Results

The automated tests cover a valid model and invalid inputs: an unsupported operator, a missing tensor, a duplicate output, incompatible matrix shapes and an invalid Softmax axis. Separate fixtures exercise constant folding, transposed Gemm, the supported operators and numerical output comparison. A fault-injection test checks that Guardian detects changes that affect output while ignoring an unreachable-node change.

Both CTest targets pass in the GitHub Windows and Linux builds. Those builds also compile the generated C++ and run its zero-input, single-input and batch CSV forms. The batch test checks output scores and rejects malformed input. A local Windows policy has occasionally blocked launching newly built executables, so the independent CI runs provide the repeatable execution evidence.

7. Progress and Remaining Work

The five core modules and a working end-to-end demonstration are complete for the .mforge path. The main remaining work is broader validation: testing more model shapes, checking generated results against an independent runtime for real ONNX files, measuring performance, and collecting stronger evidence for the Guardian probe strategy. The optional ONNX and ONNX Runtime paths require separate dependencies and are not claimed as verified by the default build.

8. Conclusion

Phase 2 demonstrates the planned compiler concepts in executable form: parsing, semantic checks, a symbol table, IR construction, optimization, target-code generation and error reporting. A student can show each intermediate result, run a valid model, explain an invalid-model diagnostic and inspect the generated C++ source. The next phase should focus on independent verification and a wider set of models rather than simply adding more operators.